

Lógica de Programação

Prof.^a Kecia Aline Marques Ferreira

DECOM | Departamento de
Computação



Lógica de Programação

Funções

Funções

- Conceito
- Forma geral de uma função
- Argumentos e parâmetros
- Retorno de função
- Exemplos

Conceito

Conceito

Considere um livro muito grande escrito sem subdivisão alguma

Isso torna o livro difícil de ler, de entender e de localizar suas partes

Um problema similar, e com consequências ainda piores, acontece com os programas

Quando um programa é escrito sem **subdivisões**, ele se torna difícil de ler e a sua manutenção torna-se complexa

Em um programa, essas subdivisões são as **funções**

Conceito

Um função é como um setor dentro do programa que possui uma tarefa específica

Uma função realiza a sua tarefa quando chamada por outra parte do programa

Ao terminar a sua tarefa, a função retorna a execução para o ponto do programa que a chamou

Conceito

Nos programas vistos até agora, foram usadas funções já implementadas na biblioteca da linguagem C

São exemplos: printf, scanf, getchar, fgets, pow, etc.

Vamos aprender como construir as suas funções nos seus programas

Um programa pode ter tantas funções quantas forem necessárias

Conceito

Todo programa em C tem pelo menos uma função: main

A execução do programa em C inicia sempre pela função main

Dentro da main, outras funções podem ser chamadas

Forma Geral de uma Função

Forma Geral de uma Função

A escrita de uma função tem a seguinte forma geral

```
tipo_do_retorno nome_da_função(lista_de_parâmetros){  
  
    corpo_da_função  
  
}
```

Forma Geral de uma Função

Onde:

tipo_do_retorno: é o tipo de dado que a função retorna como resultado da sua execução

nome_da_função: deve ser um nome significativo que represente bem a tarefa realizada pela função

lista_de_parâmetros: lista dos dados e seus respectivos tipos a serem passados para a função realizar a sua tarefa, separados por vírgulas

corpo_da_função: comandos que definem a execução da função. No corpo da função, podem ser declaradas variáveis a serem usadas dentro da função. Essas variáveis são chamadas variáveis locais.

Forma Geral de uma Função

Há situações, em que uma função realiza a sua tarefa, mas não retorna um valor ao final

Nessas situações, define-se o tipo de retorno da função como **void**

void indica ausência de valor

Parâmetros e Argumentos

Parâmetros e Argumentos

Quando uma função é **declarada**, é preciso especificar os **parâmetros** que ela recebe para realizar a sua tarefa e o tipo do dado que ela retorna como resultado

Por exemplo:

a função **double pow(double base, double exp)**

recebe dois parâmetros do tipo double (base e exp) e retorna um valor do tipo double

Parâmetros e Argumentos

No programa, quando uma função é usada, diz-se que ela está sendo **chamada**

Ao se chamar uma função, deve-se passar para ela os dados que ela espera

Na **chamada da função**, os dados que passamos para ela são denominados **argumentos**

Por exemplo, na linha de código abaixo

```
result = pow(2,i);
```

A função `pow` está sendo chamada, passando-lhe dois argumentos: 2 e o valor da variável `i`.

A função `pow` realizará a sua tarefa e retornará o valor resultante. Esse valor será armazenado na variável `result`.

Retorno de Funções

Retorno de Funções

Ao término da realização da sua tarefa, a função pode retornar um resultado para quem a chamou

Para isso, utiliza-se o comando **return**

Quando o comando `return` é executado, ele retorna o fluxo de execução do programa para o ponto seguinte ao da chamada da função

Quando a função retorna um valor, o comando **return** deve ser seguido da expressão cujo valor será retornado

Exemplos:

```
return x;
```

```
return a+b;
```

```
return -1;
```

Retorno de Funções

No caso de funções do tipo *void*, a função não retorna valor

Nesse caso:

- Não é necessário usar o comando **return**
- Se for utilizado, o comando **return** não é seguido de uma expressão

Exemplo:

```
return;
```

Exemplos

Exemplo

Um programa para ler um número inteiro positivo e imprimir os números inteiros positivos menores do que ele.

Exemplo

```
#include <stdio.h>
#include <conio.h>
```

Tipo do retorno

Parâmetros

```
void imprimeMenores(int n){
```

```
    for(int i=0; i<n; i++)
        printf("%d ", i);
```

```
}
```

```
void main(){
    int num;
    char resp;
```

```
do{
```

```
    printf("\n\nDigite um número: ");
    scanf("%d", &num);
```

Chamada da função

```
    imprimeMenores(num);
```

Argumento

```
    printf("\nDeseja continuar? (S/N) ");
    resp = getche();
```

```
}while(resp=='S' || resp=='s');
```

```
}
```

Exemplo

Um programa para ler a base e altura de retângulos e informar a sua área. O programa deverá encerrar o processamento quando o usuário informar que não deseja mais continuar.

Exemplo

```
#include <stdio.h>
#include <conio.h>
```

Tipo do retorno

Parâmetros

```
float areaRetangulo(float base, float altura){
    float area;
```

```
    if (base<=0 || altura <=0)
```

```
        area=-1;
```

```
    else
```

```
        area=base*altura;
```

```
    return area;
```

Variável local

```
}
```

Retorno

Exemplo

```
void main(){
    float base, altura, area;
    char resp;

    do{
        printf("\n\nDigite a base: ");
        scanf("%f", &base);
        printf("\nDigite a altura: ");
        scanf("%f", &altura);

        area = areaRetangulo(base, altura);

        printf("\nA area e: %.5f.", area);

        printf("\nDeseja continuar?(S/N) ");
        resp = getche();
    }while(resp=='S' || resp=='s');
}
```

Chamada da função

Argumentos

Exemplo

Um programa para calcular a área de retângulos, círculos e triângulos. O usuário deverá informar o tipo de figura geométrica. No caso de retângulo e triângulo, deverá informar também a base e altura da figura. No caso de círculo, deverá informar o raio.

O programa deverá encerrar o processamento quando o usuário informar que não deseja prosseguir.

Exemplo

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
#include <stdlib.h>

float areaRetangulo(float base, float altura){
    float area;

    if (base<=0 || altura <=0)
        area=-1;
    else
        area=base*altura;

    return area;
}
```

Exemplo

```
float areaTriangulo(float base, float altura){  
    float area;  
  
    if (base<=0 || altura <=0)  
        area=-1;  
    else  
        area=base*altura/2;  
  
    return area;  
}
```

Exemplo

```
float areaCirculo(float raio){  
    float area;  
  
    if (raio <=0)  
        area=-1;  
    else  
        area=M_PI*pow(raio,2);  
  
    return area;  
}
```

Exemplo

```
void main(){
    float base, altura, raio, area;
    char opc, resp;

    do{
        system("cls");
        printf("\n\n1 - Retangulo \n2 - Triangulo \n3 - Circulo")        ;
        printf("\n\nDigite a opcao: ");
        opc = getche();

        switch(opc){
            case '1': printf("\n\nDigite a base: ");
                      scanf("%f", &base);
                      printf("\nDigite a altura: ");
                      scanf("%f", &altura);
                      area = areaRetangulo(base, altura);
                      printf("\nA area do retangulo e: %.5f.", area);
                      break;
```

Exemplo

```
case '2': printf("\n\nDigite a base: ");
          scanf("%f", &base);
          printf("\nDigite a altura: ");
          scanf("%f", &altura);
          area = areaTriangulo(base, altura);
          printf("\nA area do triangulo e: %.5f.", area);
          break;

case '3': printf("\n\nDigite o raio: ");
          scanf("%f", &raio);
          area = areaCirculo(raio);
          printf("\nA area do circulo e: %.5f.", area);
          break;

default:  printf("\nOpcao invalida.");
}
```

Exemplo

```
printf("\nDeseja continuar?(S/N) ");  
resp = getche();  
  
}while(resp=='S' || resp=='s');  
  
}
```